Class :10 Web Scraping

Heart Disease prediction ¶

age: Person,s age

sex: Person,s Gender

cp: Chest pain of four types (1_4)

trestbps: person,s Restig Blood Pressure

Chol: Cholestrolelvl

Fbs: Fastind blood sugar level (0_2)

restecg(0_2)

thalach: maximum heart rate achieved

exang

oldpeak

slope

ca

thal(3: Normal, 6= fixed , 7 = reverseable)

terget : 0 = Np heart disease -1 = yes

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
data =  pd.read_csv("heart.csv")
data
```

Out[2]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1020	59	1	1	140	221	0	1	164	1	0.0	2	0	2	1
1021	60	1	0	125	258	0	0	141	1	2.8	1	1	3	0
1022	47	1	0	110	275	0	0	118	1	1.0	1	1	2	0
1023	50	0	0	110	254	0	0	159	0	0.0	2	0	2	1
1024	54	1	0	120	188	0	1	113	0	1.4	1	1	3	0

1025 rows × 14 columns

In [3]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       1025 non-null   int64  
 1   sex       1025 non-null   int64  
 2   cp        1025 non-null   int64  
 3   trestbps  1025 non-null   int64  
 4   chol      1025 non-null   int64  
 5   fbs       1025 non-null   int64  
 6   restecg   1025 non-null   int64  
 7   thalach   1025 non-null   int64  
 8   exang     1025 non-null   int64  
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64  
 11  ca        1025 non-null   int64  
 12  thal      1025 non-null   int64  
 13  target    1025 non-null   int64  
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

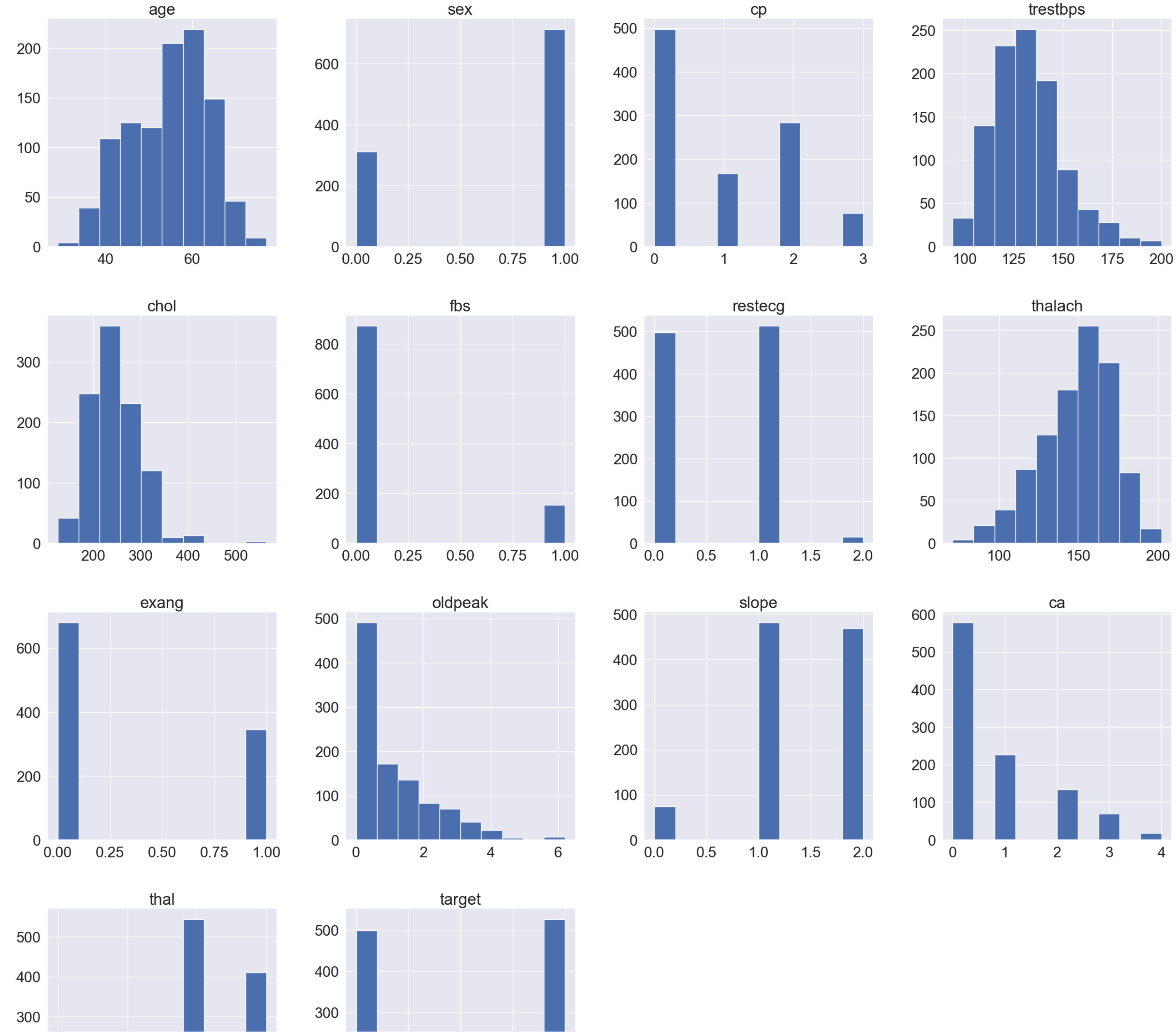
In [4]:
```
data.describe()
```

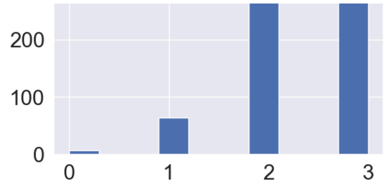
Out[4]:

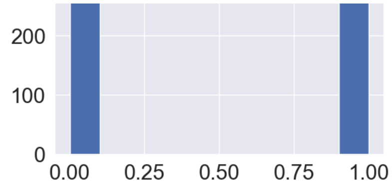
	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpea
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.00000	1025.000000	1025.000000	1025.000000	1025.000000	1025.00000
mean	54.434146	0.695610	0.942439	131.611707	246.00000	0.149268	0.529756	149.114146	0.336585	1.07151
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	0.527878	23.005724	0.472772	1.17505
min	29.000000	0.000000	0.000000	94.000000	126.00000	0.000000	0.000000	71.000000	0.000000	0.00000
25%	48.000000	0.000000	0.000000	120.000000	211.00000	0.000000	0.000000	132.000000	0.000000	0.00000
50%	56.000000	1.000000	1.000000	130.000000	240.00000	0.000000	1.000000	152.000000	0.000000	0.80000
75%	61.000000	1.000000	2.000000	140.000000	275.00000	0.000000	1.000000	166.000000	1.000000	1.80000
max	77.000000	1.000000	3.000000	200.000000	564.00000	1.000000	2.000000	202.000000	1.000000	6.20000

In [5]:
```
sns.set(font_scale = 2)
data.hist(figsize = (30,30))
plt.show
```

Out[5]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```


age
0 50 100 150 200
40 60
sex
0 200 400 600
0.00 0.25 0.50 0.75 1.00
cp
0 100 200 300 400 500
0 1 2 3
trestbps
0 50 100 150 200 250
100 125 150 175 200
chol
0 100 200 300
200 300 400 500
fbs
0 200 400 600 800
0.00 0.25 0.50 0.75 1.00
restecg
0 100 200 300 400 500
0.0 0.5 1.0 1.5 2.0
thalach
0 50 100 150 200 250
100 150 200
exang
0 200 400 600
0.00 0.25 0.50 0.75 1.00
oldpeak
0 100 200 300 400 500
0 2 4 6
slope
0 100 200 300 400 500
0.0 0.5 1.0 1.5 2.0
ca
0 100 200 300 400 500 600
0 1 2 3 4
thal
300 400 500
target
300 400 500



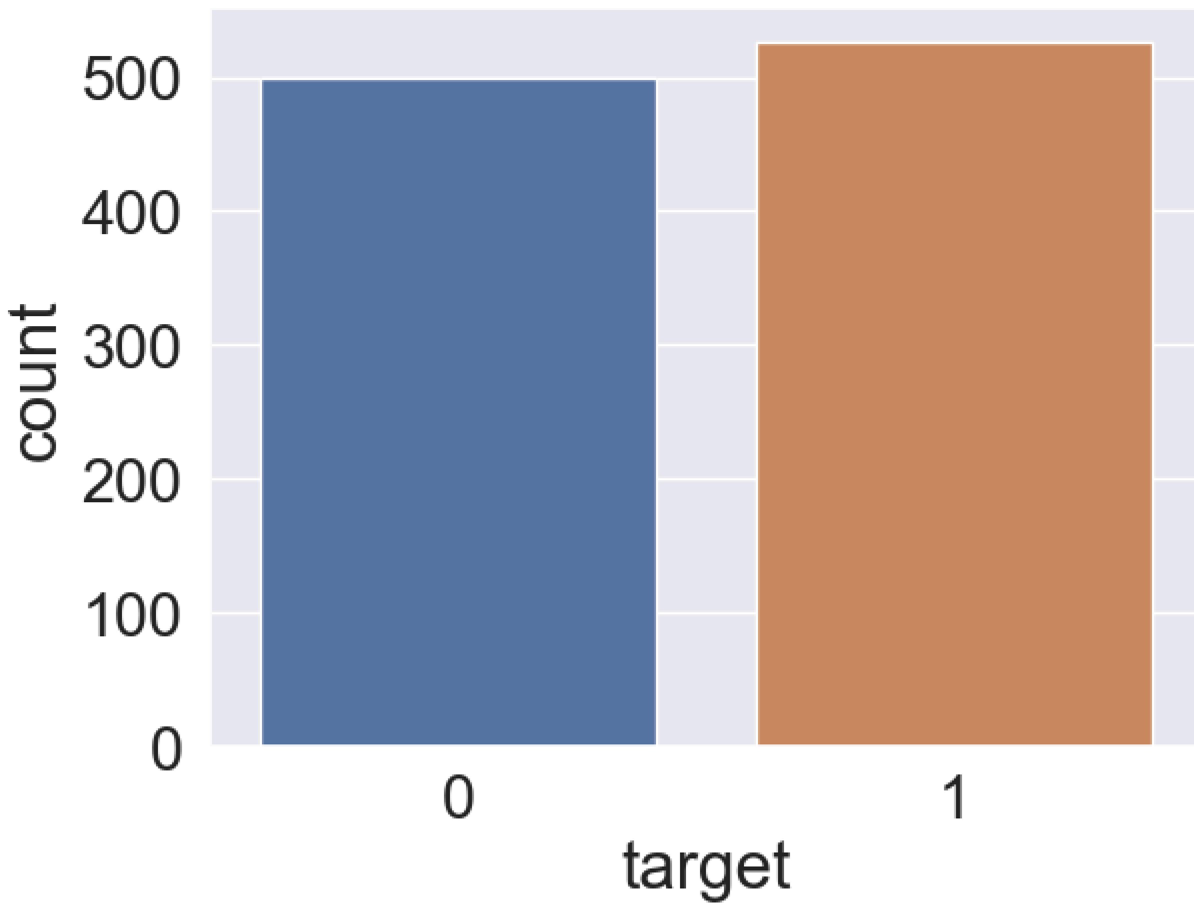


In [6]:
```
data["target"].value_counts()
```

Out[6]:
```
target
1    526
0    499
Name: count, dtype: int64
```

In [7]:
```
sns.countplot(x = "target", data = data)
```

Out[7]:
```
<Axes: xlabel='target', ylabel='count'>
```

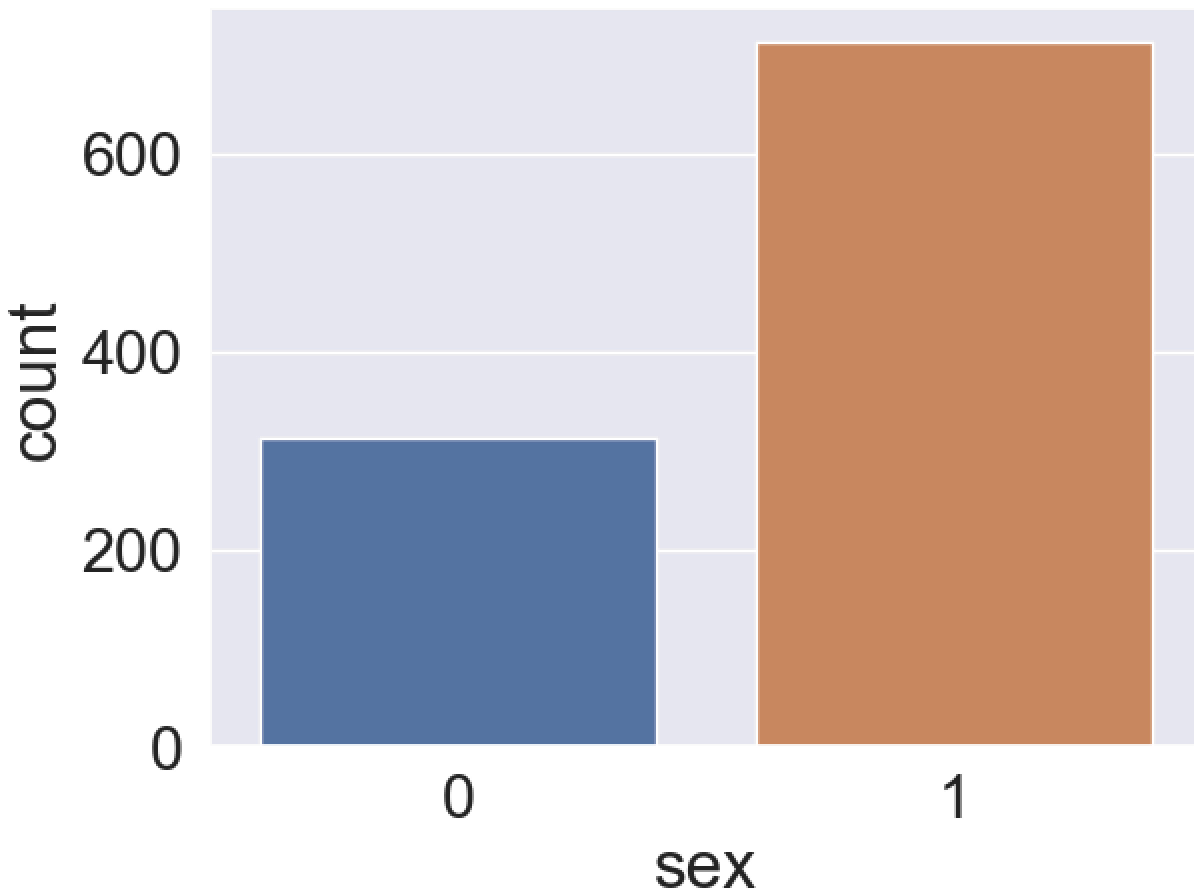


In [8]:
```
data["sex"].value_counts()
```

Out[8]:
```
sex
1    713
0    312
Name: count, dtype: int64
```

In [9]:
```
sns.countplot( x = "sex", data = data)
```

Out[9]:
```
<Axes: xlabel='sex', ylabel='count'>
```

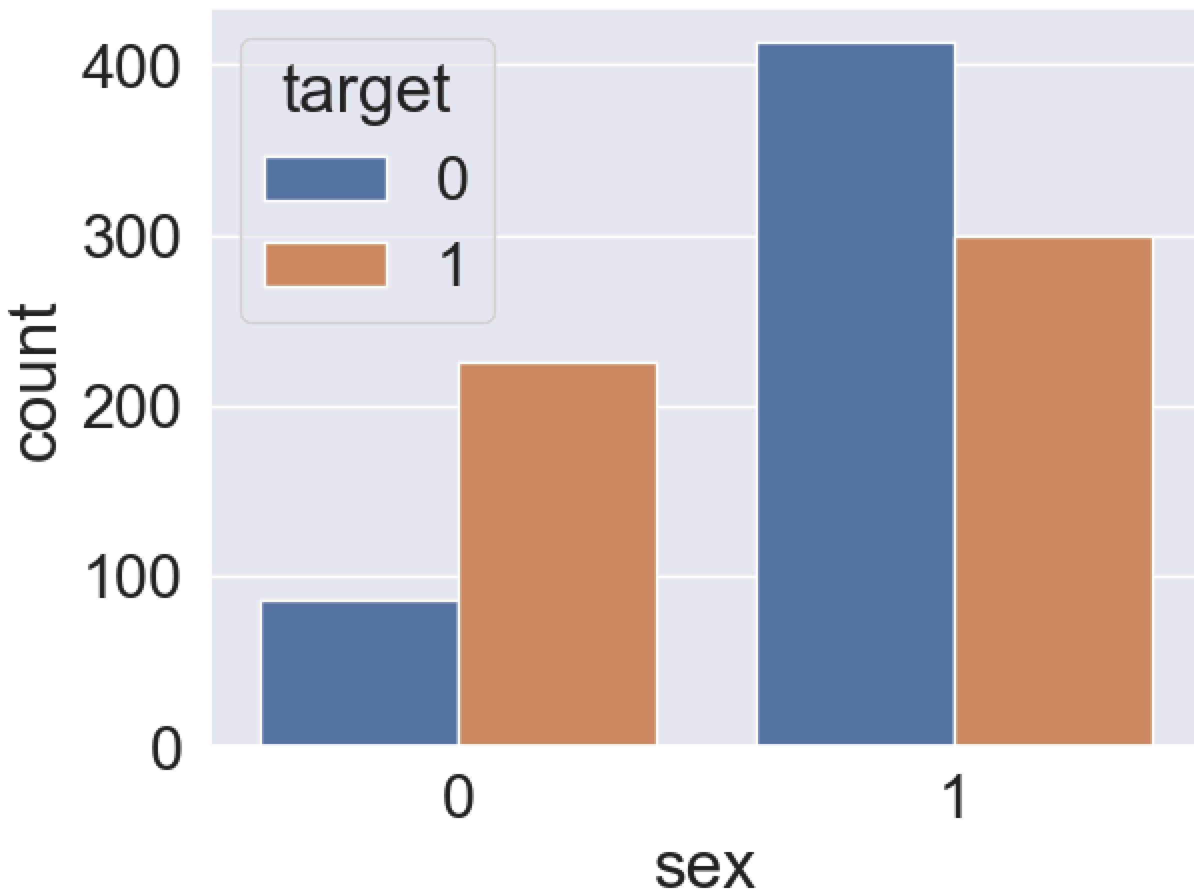


In [10]:
```
data.groupby("sex")["target"].value_counts()
```

Out[10]:
```
sex  target
0    1         226
     0          86
1    0         413
     1         300
Name: count, dtype: int64
```

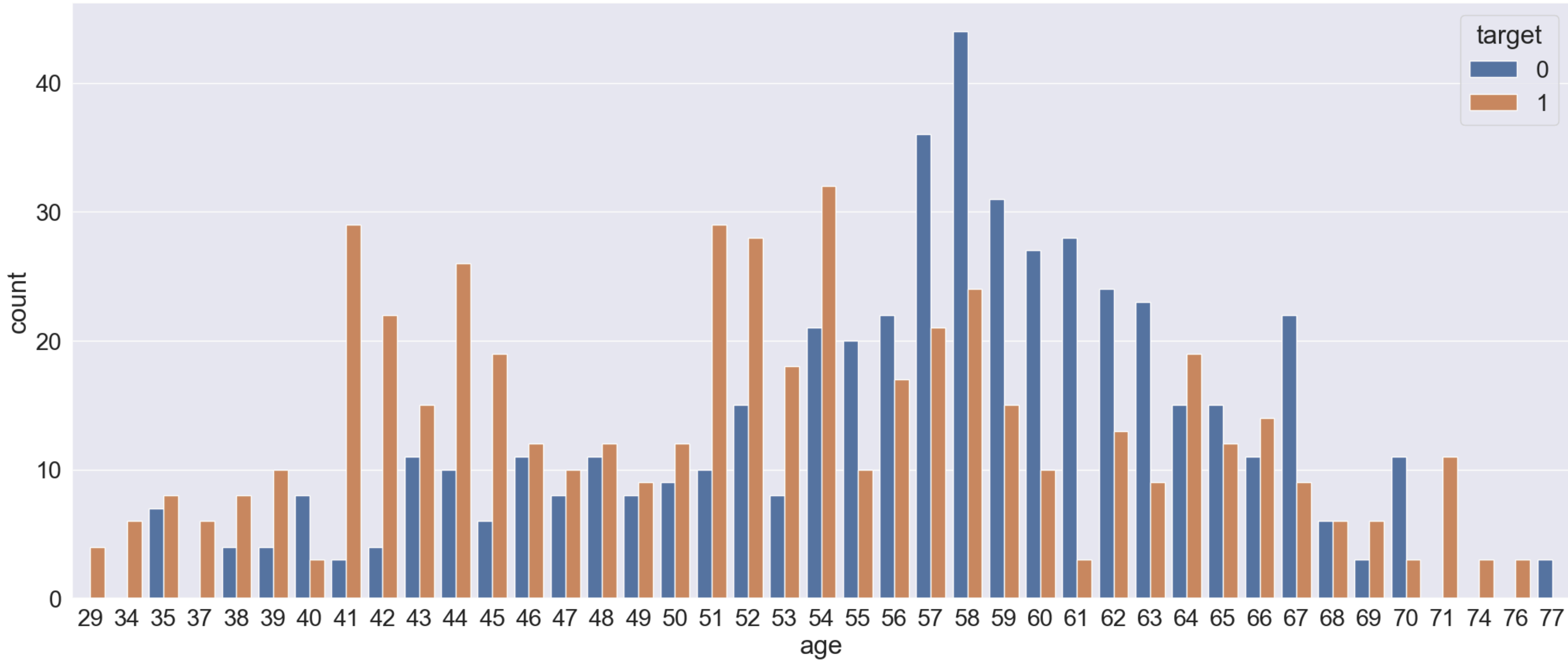
In [11]:
```
sns.countplot(x = "sex", hue = "target", data = data)
```

Out[11]: <Axes: xlabel='sex', ylabel='count'>



In [12]:
```
plt.figure(figsize = ( 25 , 10))
sns.countplot(x = "age", hue = "target", data = data)
```

Out[12]: <Axes: xlabel='age', ylabel='count'>



In [15]:
```
data.columns
```

Out[15]:
```
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],
      dtype='object')
```

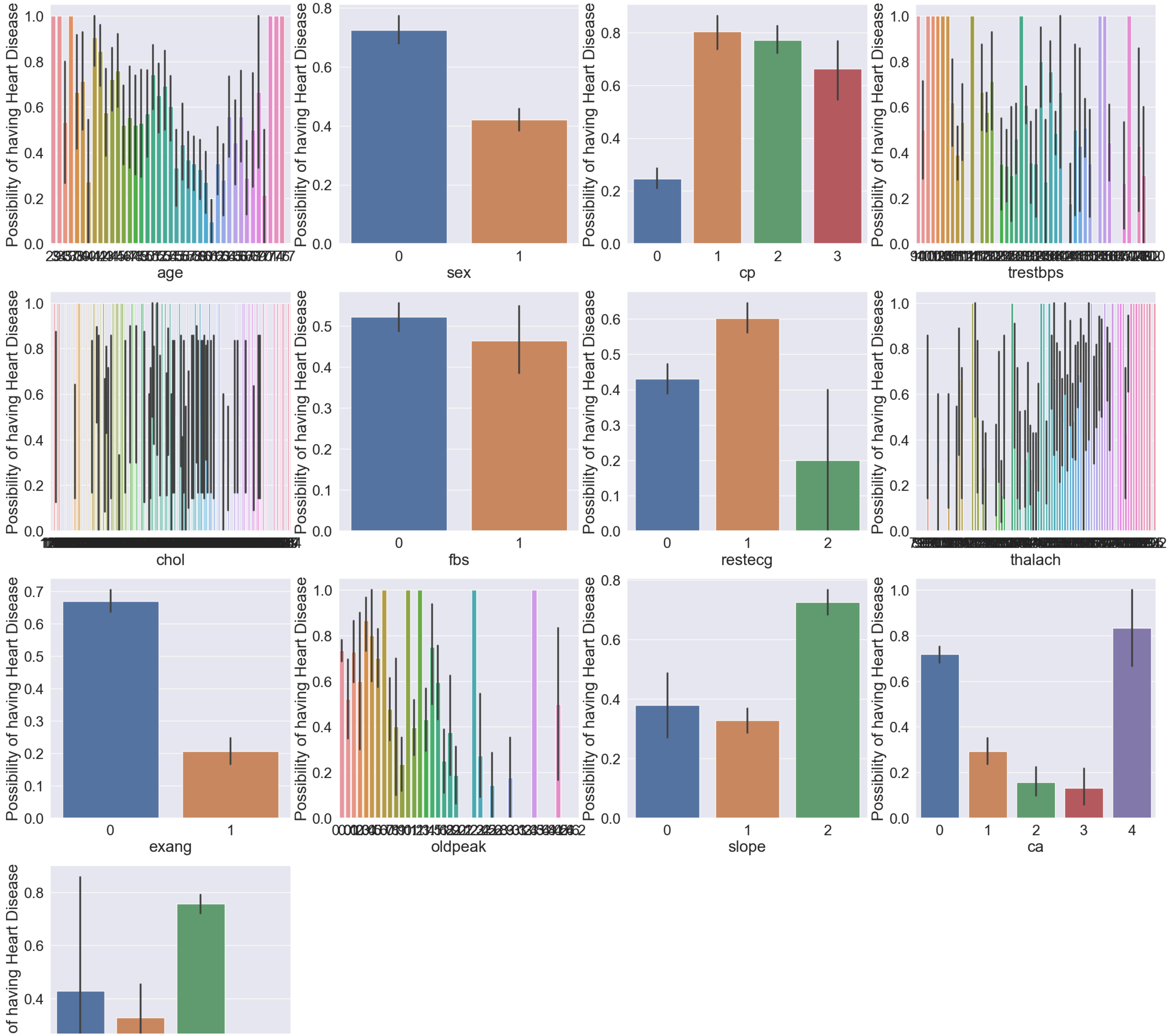
In [16]:
```
categorical_values = list (data.columns)
```

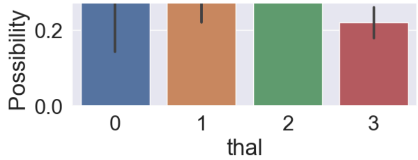
In [17]:
```
categorical_values.remove("target")
```

In [18]:
```
print(categorical_values)
```

```
['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'th
al']
```

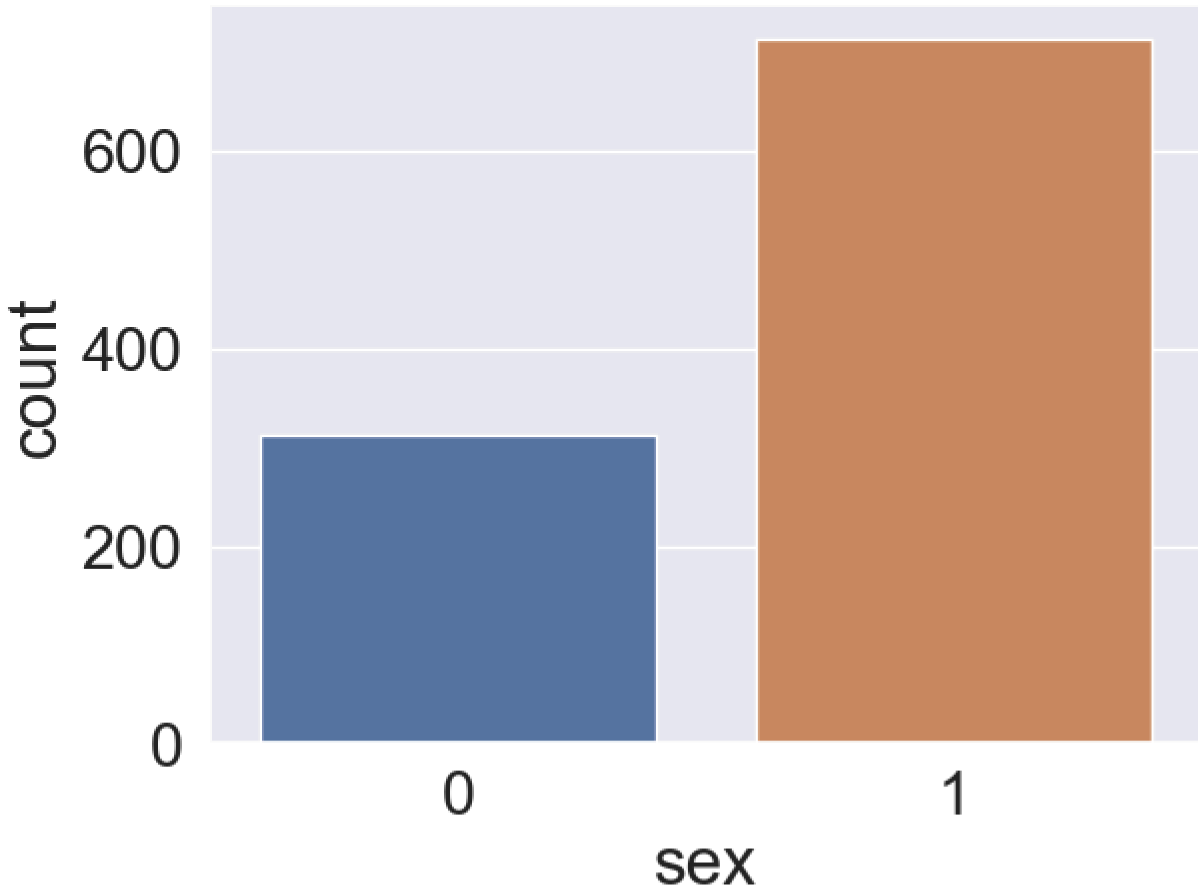
In [19]:
```
plt.figure(figsize=(30,30))

for i, col in enumerate(categorical_values, 1):
    plt.subplot(4,4,i)
    sns.barplot(x=f"{col}",y='target', data=data)
    plt.ylabel("Possibility of having Heart Disease")
    plt.xlabel(f"{col}")
```


Possibility of having Heart Disease
age
sex
cp
trestbps
chol
fbs
restecg
thalach
exang
oldpeak
slope
ca
of having Heart Disease



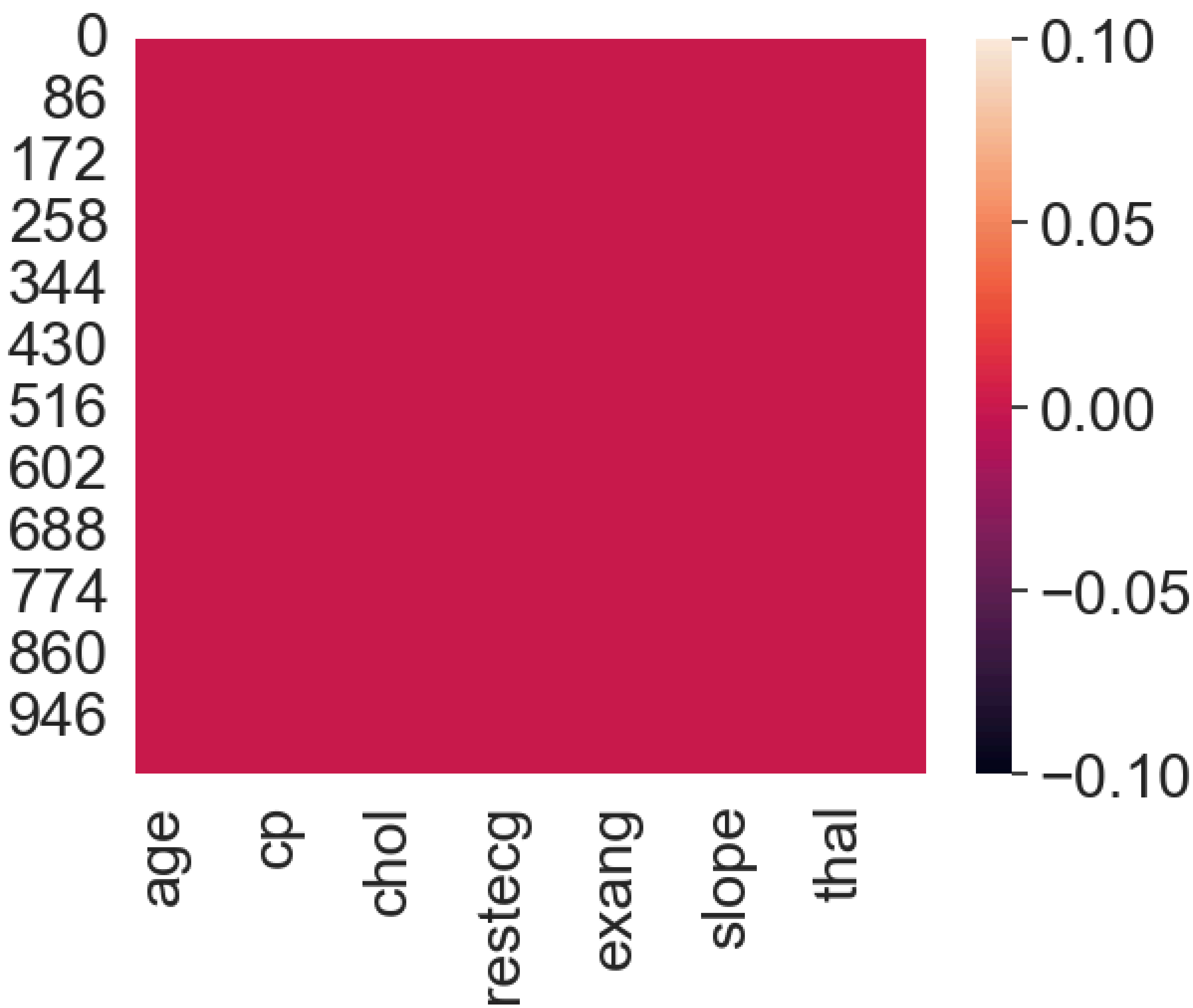
In [20]:
```
sns.countplot(x = "sex", data = data)
```

Out[20]: <Axes: xlabel='sex', ylabel='count'>



In [21]:
```
sns.heatmap(data.isnull())
```

Out[21]: <Axes: >



In [22]:
```
data.shape
```

Out[22]: (1025, 14)

In [23]:
```
data.info
```

Out[23]:
```
<bound method DataFrame.info of       age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  \
0      52    1   0       125   212    0        1      168      0      1.0
1      53    1   0       140   203    1        0      155      1      3.1
2      70    1   0       145   174    0        1      125      1      2.6
3      61    1   0       148   203    0        1      161      0      0.0
4      62    0   0       138   294    1        1      106      0      1.9
...   ...  ...  ..       ...   ...  ...      ...      ...    ...      ...
1020   59    1   1       140   221    0        1      164      1      0.0
1021   60    1   0       125   258    0        0      141      1      2.8
1022   47    1   0       110   275    0        0      118      1      1.0
1023   50    0   0       110   254    0        0      159      0      0.0
1024   54    1   0       120   188    0        1      113      0      1.4

      slope  ca  thal  target
0         2   2     3       0
1         0   0     3       0
2         0   0     3       0
3         2   1     3       0
4         1   3     2       0
...     ...  ..   ...     ...
1020      2   0     2       1
1021      1   1     3       0
1022      1   1     2       0
1023      2   0     2       1
1024      1   1     3       0

[1025 rows x 14 columns]>
```

In [24]:
```
data.head()
```

Out[24]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

In [25]:
```
data.describe()
```

Out[25]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpea
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.00000	1025.000000	1025.000000	1025.000000	1025.000000	1025.00000
mean	54.434146	0.695610	0.942439	131.611707	246.00000	0.149268	0.529756	149.114146	0.336585	1.07151
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	0.527878	23.005724	0.472772	1.17505
min	29.000000	0.000000	0.000000	94.000000	126.00000	0.000000	0.000000	71.000000	0.000000	0.00000
25%	48.000000	0.000000	0.000000	120.000000	211.00000	0.000000	0.000000	132.000000	0.000000	0.00000
50%	56.000000	1.000000	1.000000	130.000000	240.00000	0.000000	1.000000	152.000000	0.000000	0.80000
75%	61.000000	1.000000	2.000000	140.000000	275.00000	0.000000	1.000000	166.000000	1.000000	1.80000
max	77.000000	1.000000	3.000000	200.000000	564.00000	1.000000	2.000000	202.000000	1.000000	6.20000

In [ ]: